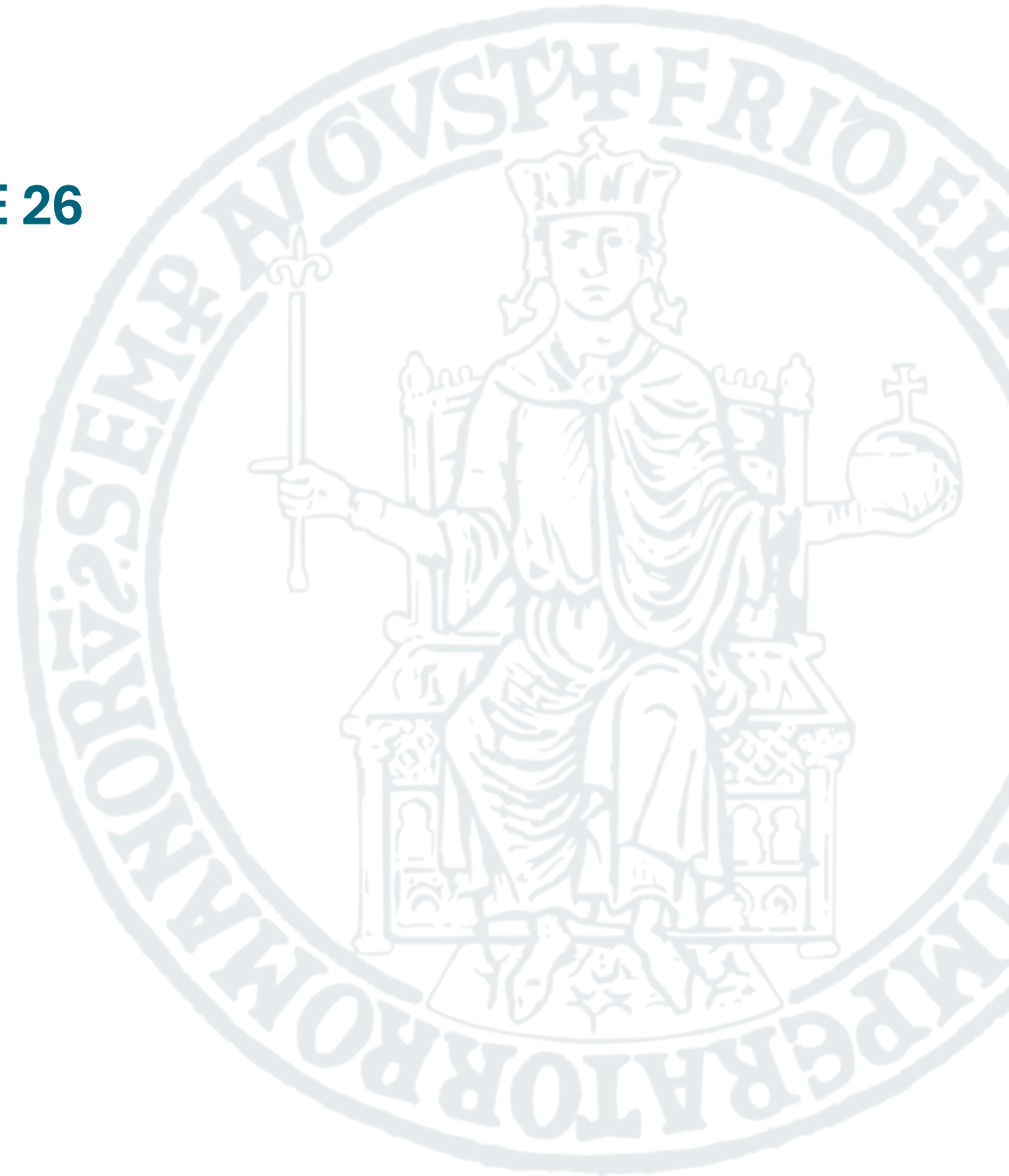


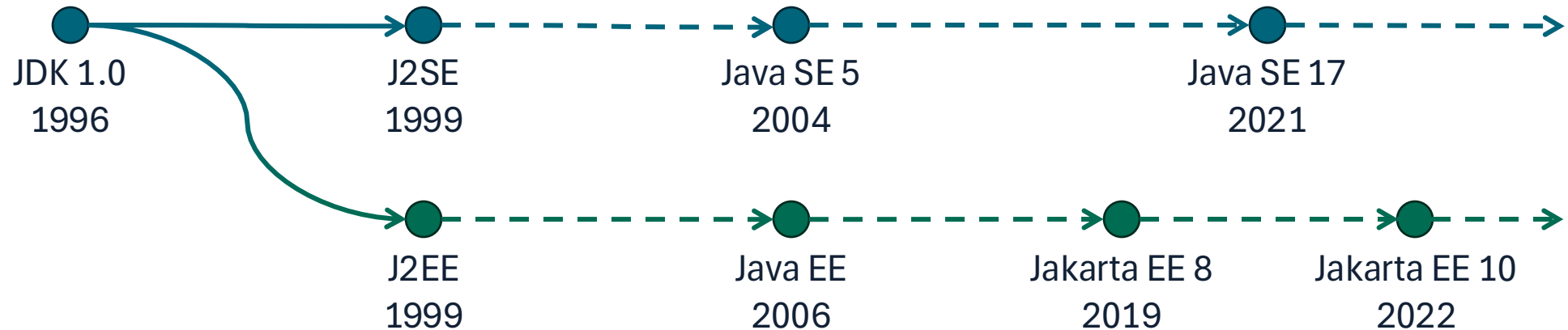
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
SOFTWARE ENGINEERING – LECTURE 26

Implementing a REST API software with Jakarta EE (JAX-RS)

Prof. Bernardo Breve



JAKARTA ENTERPRISE EDITION (EE)



- Jakarta EE is evolution of Java EE
- It's a set of specification, including standards such as Servlets, Jakarta Server Pages, **JAX-RS**...
- **JAX-RS** is a specification for implementing **REST API**

JAX-RS AND ITS IMPLEMENTATIONS

- JAX-RS is just a set of specifications
- The most well-known implementations are **frameworks** such as
 - [Eclipse Jersey](#) (we're gonna use this in the demo!)
 - [JBoss RESTEasy](#)
- Annotation-based: specific Java annotations should be used to «tell» the frameworks when to run the code we write

TODOs Endpoints

- **POST /auth**

→ *Input:* { username, password } → *Output:* JWT token

- **GET /greet**

→ *Input:* Authorization: Bearer <token> → *Output:* greeting message (plain text)

- **GET /todos**

→ *Input:* Authorization: Bearer <token> → *Output:* list of todos (JSON)

- **POST /todos**

→ *Input:* { title, description }, Authorization: Bearer <token> → *Output:* created todo (JSON)

- **DELETE /todos/{id}**

→ *Input:* Authorization: Bearer <token> → *Output:* status (204 No Content)

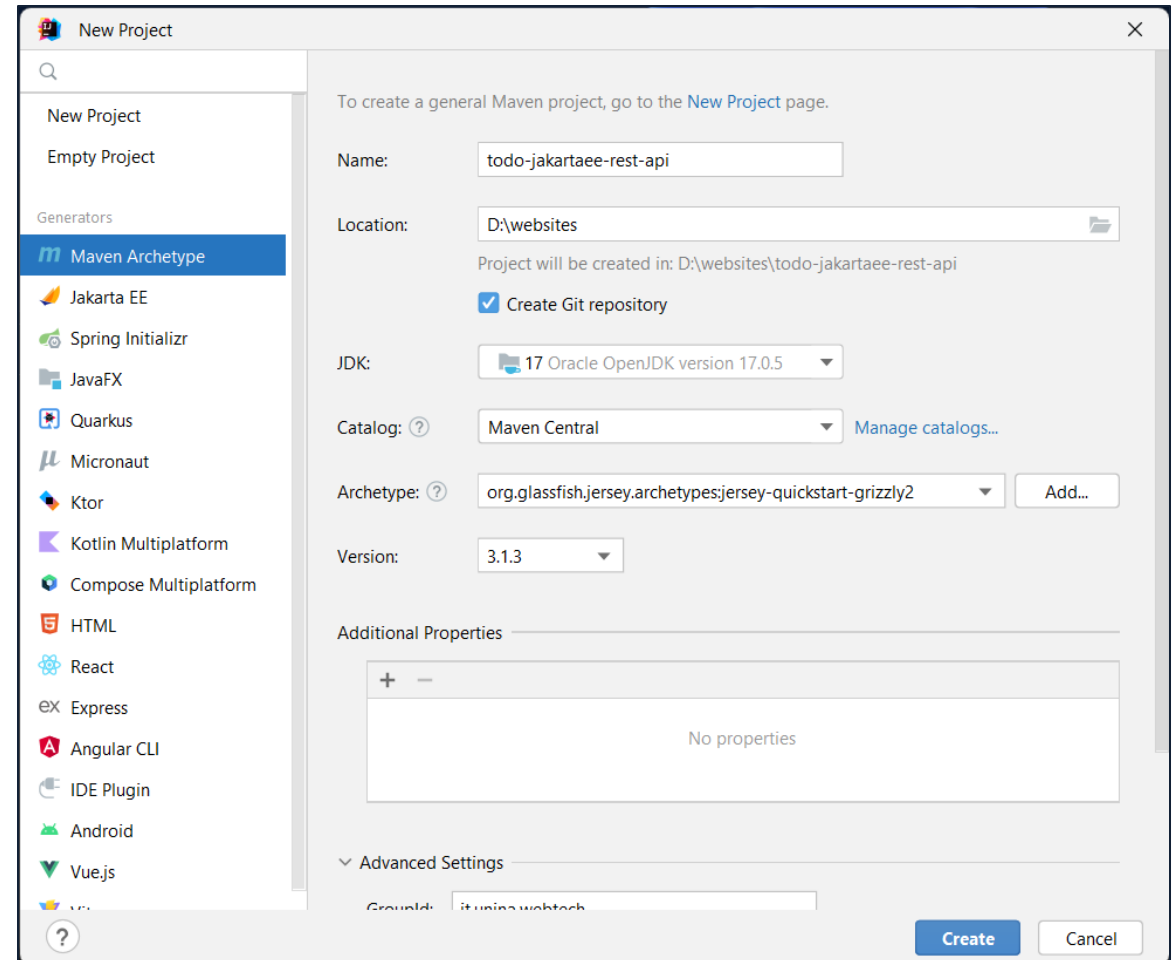
REST API USING JAX-RS

- Let's take a look at the code
- <https://github.com/luistar/jakartaee-rest-example>



MAVEN ARCHETYPE

- The easiest way to get started is to use a **Maven Archetype**
- Archetypes are basically templates, including dependencies and boilerplate code
- We'll use the **jersey-quickstart-grizzly2** archetype from Maven Central



ADD SUPPORT FOR JSON

- After creating your project from the archetype, remember to include JSON support by uncommenting the following dependency in the **pom.xml**

```
<dependency>  
    <groupId>org.glassfish.jersey.media</groupId>  
    <artifactId>jersey-media-json-binding</artifactId>  
</dependency>
```

CODE EXAMPLES

```
@Path("greet") //this class handles requests on the /greet path
public class GreeterController {
    @GET //when a request is made using the GET verb, invoke this method
    @Produces(MediaType.TEXT_PLAIN) //map return value to plaintext
    public String greet(){
        return "Hello User!";
    }
}
```

CODE EXAMPLES

```
@Path("/todos")
public class TodoController {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<TodoItem> getAll() {
        return TodoItemDAO.getAll();
    }
}
```

- The returned `List<TodoItem>` is automatically converted to a JSON representation (thanks to the `@Produces` annotation)!

CODE EXAMPLES

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response addTodoItem(TodoItem todo){
    try {
        TodoItemDAO.add(todo);
        return Response.status(Response.Status.CREATED).build();
    } catch (Exception e) {
        return Response.status(Response.Status.BAD_REQUEST)
            .entity(e.getMessage()).build();
    }
}
```

- The JSON object received in the body of the **@POST** request is automatically serialized in the `TodoItem` argument (thanks to the **@Consumes** annotation)

OpenAPI



OPENAPI: DESCRIBING A REST API

- [OpenAPI](#) (formerly Swagger) is an open, formal standard to describe HTTP APIs
- Why?
 - **Standardization:** ensures consistent documentation practices
 - **Documentation:** can be used to automatically generate comprehensive docs
 - **Code generation:** Allows for automatic generation of client libraries and server stubs
 - **Machine and human-readable:** enables automated discovery and more
 - **Most broadly adopted industry standard**



THE OPENAPI SPECIFICATION

- **OpenAPI Descriptions** are written as structured text documents
- Each document represents a JSON object, in JSON or [YAML](#) format
- These specification files describe version of the OpenAPI specification (**openapi**), the API (**info**) and the endpoints (**paths**)

```
# YAML format
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {} # No endpoints defined
```

```
// JSON format
{
  "openapi": "3.1.0",
  "info": {
    "title": "A minimal specification",
    "version": "0.0.1"
  },
  "paths": {} // No endpoints defined
}
```

THE OPENAPI SPECIFICATION: PATHS

- The API Endpoints are called **Paths** in the OpenAPI specification
- **Path** objects allow to specify a sequence of endpoints
 - For each endpoint, its supported methods
 - For each method,
 - Expected input parameters (if any) (e.g.: path parameters, headers, etc...)
 - Request body (if any)
 - Possible responses
 - And more!

THE OPENAPI SPECIFICATION: PATHS

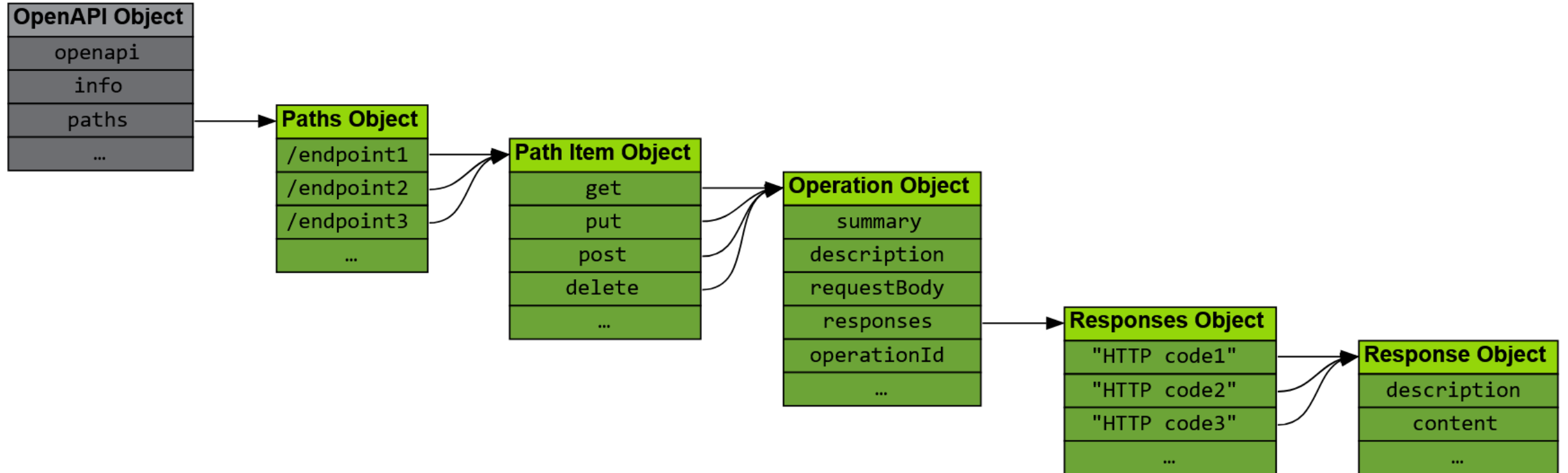


Image from <https://learn.openapis.org/specification/paths>

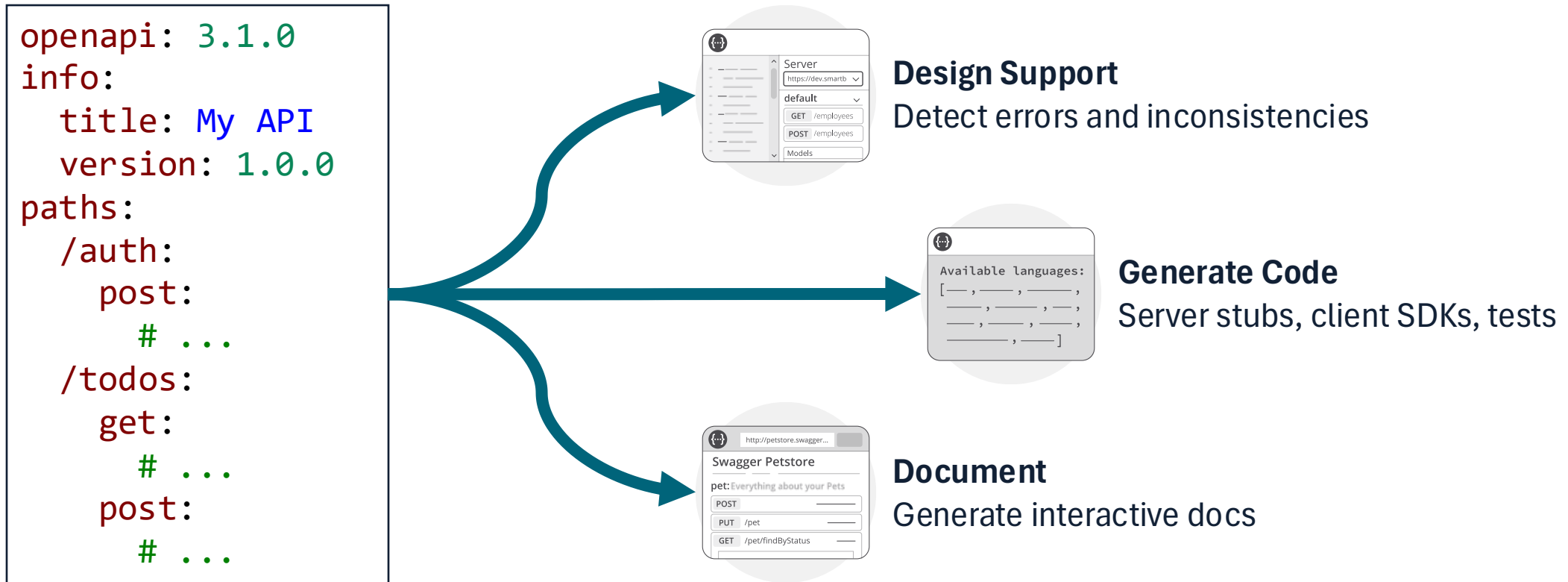
PATH EXAMPLE

- /auth endpoint, POST method
- Expects Request Body containing a JSON object with type {usr: string, pwd: string}
- Returns 200 OK on success, 401 Unauthorized when credentials are invalid

```
/auth:
  post:
    description: Authenticate user
    produces:
      - application/json
    requestBody:
      description: user to authenticate
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              usr:
                type: string
                example: Kyle
              pwd:
                type: string
                example: p4ssw0rd
    responses:
      200:
        description: User authenticated
      401:
        description: Invalid credentials
```

WORKING WITH OPENAPI SPECIFICATIONS

- Writing an OpenAPI specification for an API is quite a lot of work
- What can we do with an OpenAPI specification?



OPENAPI: OPEN SOURCE TOOLS

- OpenAPI is supported by a number of mature, open-source tools
- Some widely used open-source tools are reported as follows:
 - **Swagger Editor:** <https://swagger.io/tools/swagger-editor/>
 - **Swagger Codegen:** <https://swagger.io/tools/swagger-codegen/>
 - **Swagger UI:** <https://swagger.io/tools/swagger-ui/>
- You can also check them out in a web browser, without downloading anything: <https://editor.swagger.io/>

OPENAPI: SWAGGER EDITOR

The image shows the Swagger Editor interface. On the left, the OpenAPI specification is displayed in a code editor. The specification defines a `pet` resource with a `PUT` operation for updating an existing pet. The operation includes tags, summary, description, operationId, requestBody, and responses. The requestBody is defined with multiple media types (application/json, application/xml, application/x-www-form-urlencoded) and a schema that references `#/components/schemas/Pet`. The responses include a `200` status for a successful operation and a `400` status for a bad request.

```
40 paths:
41   /pet:
42     put:
43       tags:
44         - pet
45       summary: Update an existing pet
46       description: Update an existing pet by Id
47       operationId: updatePet
48       requestBody:
49         description: Update an existent pet in the store
50         content:
51           application/json:
52             schema:
53               $ref: '#/components/schemas/Pet'
54           application/xml:
55             schema:
56               $ref: '#/components/schemas/Pet'
57           application/x-www-form-urlencoded:
58             schema:
59               $ref: '#/components/schemas/Pet'
60         required: true
61       responses:
62         '200':
63           description: Successful operation
64           content:
65             application/json:
66               schema:
67                 $ref: '#/components/schemas/Pet'
68             application/xml:
69               schema:
70                 $ref: '#/components/schemas/Pet'
71         '400':
72           description: Invalid ID supplied
```

On the right, the Swagger UI documentation is displayed. It shows the `pet` resource with the following operations:

- PUT** `/pet` Update an existing pet
- POST** `/pet` Add a new pet to the store
- GET** `/pet/findByStatus` Finds Pets by status
- GET** `/pet/findByTags` Finds Pets by tags
- GET** `/pet/{petId}` Find pet by ID
- POST** `/pet/{petId}` Updates a pet in the store with form data
- DELETE** `/pet/{petId}` Deletes a pet
- POST** `/pet/{petId}/uploadImage` uploads an image

Specification

Documentation (Swagger UI)

OPENAPI: CODEGEN

- From the editor toolbar, we can also generate Server and Client code
- Server-side code generation can obviously only provide stubs
 - Supported languages/frameworks include: ASP.NET, Go, Java, JaxRS, Micronaut, Kotlin, Node.js, Python (Flask), Scala, Spring.
- Client-side code generation is a very handy way to make SDKs for our APIs available in tens of different languages
 - Supported languages include: C#, Dart, Go, Java, JavaScript, Kotlin, PHP, Python, R, Ruby, Scala, Swift, TypeScript

OPENAPI-DRIVEN DEVELOPMENT

- Often, developers start working on an API by defining its specification before writing any actual code
- This approach is also referred to as **OpenAPI-driven development**
- This way, they can exploit code generation capabilities to the fullest
- The approach also promotes **independence** between the different teams involved in a project (front-end, back-end, QA). The API definition keeps all these stakeholders aligned

REFERENCES (1/2)

- **The Little Book on REST Services**

By Kenneth Lange

Freely available at <https://www.kennethlange.com/books/The-Little-Book-on-REST-Services.pdf>

- **API design guide**

Google Cloud

<https://cloud.google.com/apis/design>

Relevant parts: Resource-oriented design, Resource names.

- **REST API Checklist**

By Kenneth Lange

<https://kennethlange.com/rest-api-checklist/>



REFERENCES (2/2)

- **OpenAPI**

Official Website

<https://www.openapis.org/>

<https://learn.openapis.org/> (Recommended reads from this link: Getting started, Introduction)

<https://spec.openapis.org/oas/v3.1.0> (Full specification for OpenAPI 3.1.0)

⚠ For the Web Technologies course, you need to know what the OpenAPI specification standard is, why it has been introduced, and what it can be used for. You are **not** required to learn how to write OpenAPI specification files from scratch.

- **API Documentation: The Secret to a Great API Developer Experience**

Ebook by Smartbear Software

Available at <https://swagger.io/resources/ebooks/api-documentation-the-secret-to-a-great-api-developer-experience> and archived [here](#).

⚠ Interesting read. **Not** required for the Web Technologies course, but it can be useful to better understand the challenges in designing and documenting an API.

EXAMPLES WITH DIFFERENT FRAMEWORKS

- **To-do List REST API using JakartaEE (Java)**

By [Luigi Libero Lucio Starace](#)

Includes source code, Dockerfile and instructions.

<https://github.com/luistar/jakartaee-rest-example>

- **Pizza Shop REST API using Spring (Java)**

By [Luigi Libero Lucio Starace](#)

<https://github.com/luistar/rest-service-pizzashop>

- **Quiz REST API using FastAPI (Python)**

By Márcio Lemos

<https://github.com/marciovrl/fastapi>

